

NAPCO Nucleus: An Autonomous Intelligent Orchestrator

The Digital Deputy Framework for Enterprise QA & Project Governance

Developed & Engineered by:

Mohammad Kamrul Hasan

AI-Augmented QA Architect | Adaptive Enterprise Limited

WHAT THIS DOCUMENT IS

End-to-end technical reference for the NAPCO Nucleus AI agent. Covers the architectural pattern, the universal execution lifecycle, the Python orchestrator, the Claude Agent SDK integration, the MCP tool surface, the SQLite memory layer, every third-party integration, the CI/CD execution model, the 9 workflows, recipes for adding new workflows and tools, plus concrete benefits and demerits. Written for engineers who will operate, debug, or extend the system.

9	31	6	8	~3.6k
WORKFLOWS	MCP TOOLS	TOOL MODULES	PROMPTS	PYTHON LOC

EXECUTIVE THESIS

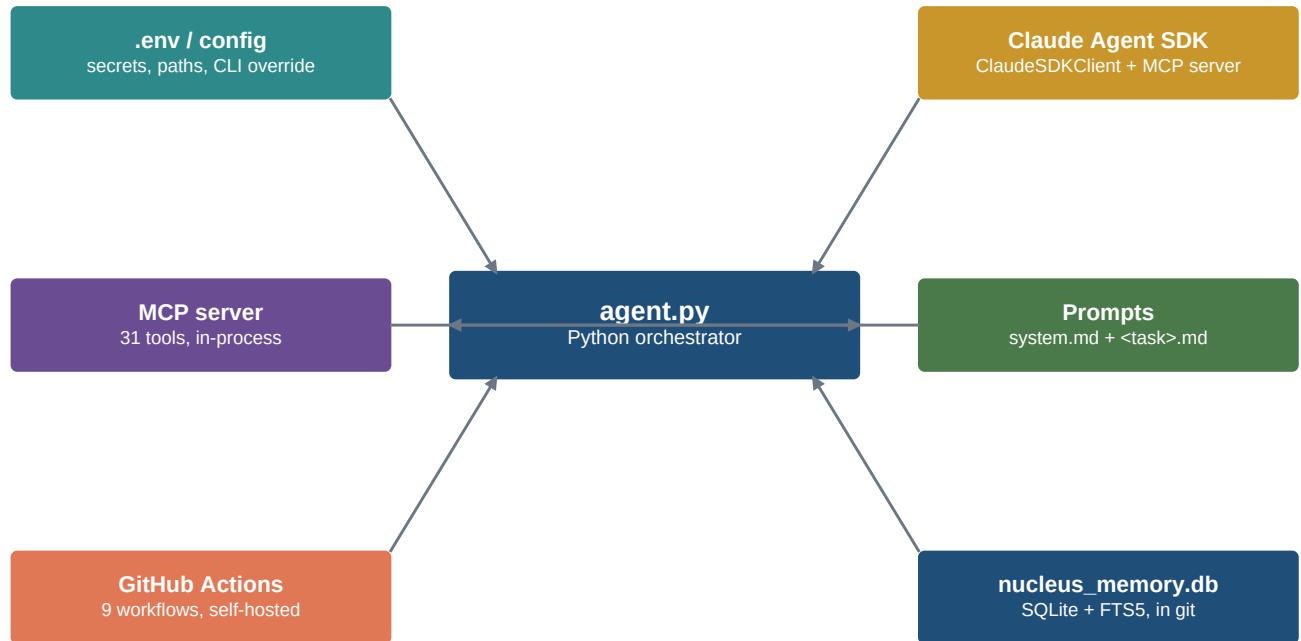
Reasoning lives in prompts. Tools wrap I/O. The Python code in NN does no algorithmic work. It exposes capabilities (IMAP poll, GitLab create issue, run Locust tier, send PDF email, etc.) and Claude orchestrates them by reading a task-specific prompt. To change behavior, you edit a markdown file. To add capability, you wrap one external system as a tool.

One Claude turn per process. agent.py boots, runs ONE turn through the SDK, exits. No daemons, no orchestration framework, no multi-agent graph. State persists between runs via SQLite committed to git, not via a long-running process.

Single-source secrets. One .env at the project root. No ANTHROPIC_API_KEY anywhere — reasoning runs through the local Claude Code CLI under a Claude Max subscription, so monthly cost is fixed.

1. Architecture

COMPONENT MAP

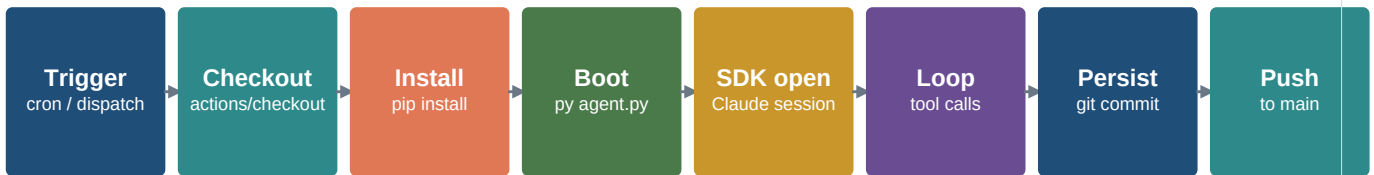


DESIGN CONSTRAINTS THAT SHAPED THE CODEBASE

- **One agent, many prompts.** Not a multi-agent graph. The same agent.py loads a different prompt per workflow.
- **Tools must be safe to call repeatedly.** Idempotency is enforced inside each tool (UID checkpoints, Drive file-ID tracking, GitLab title dedup, fuzzy memory dedup).
- **Memory survives the process.** The agent has no in-RAM state. Anything that should persist gets written to SQLite mid-run; the runner commits the DB back to git after every workflow.
- **Prompts are human-editable contracts.** Anyone can change the loop in a workflow by editing markdown. Code review for prompt changes is a markdown diff.
- **Tools never reason.** Every algorithmic decision (which requirements are real, how to split into 3-hour tasks, why a test failure is flaky vs. real, how to phrase a regression summary) lives in a prompt. Python only does I/O.

2. End-to-end execution lifecycle

THE UNIVERSAL LIFECYCLE



Universal lifecycle — every workflow follows this shape

WHAT HAPPENS IN EACH PHASE

1

Trigger fires

GitHub Actions schedule (cron) or workflow_dispatch starts the job. The job is pinned to the self-hosted Windows runner via runs-on: [self-hosted, Windows].

2

Repo checkout

actions/checkout@v5 clones the repo onto the runner. Includes the latest nucleus_memory.db and data/ committed by prior runs — that's how memory crosses runs.

3

Dependencies

py -3 -m pip install -r requirements.txt installs claude-agent-sdk, requests, google-api-python-client, pypdf, python-dotenv. Fast on warm runner.

4

agent.py boots

Loads .env at project root with override=True so .env wins over empty workflow env vars. Forces UTF-8 stdout/stderr to survive Windows cp1252 console.

5

Claude SDK session opens

create_sdk_mcp_server registers all 31 tools as 'napco-nucleus'. ClaudeSDKClient opens a session against the local Claude Code CLI (no API key). System prompt = system.md + .md.

6

The loop runs

client.query(kickoff) sends the task-specific kickoff message. Claude reads the prompt, calls MCP tools as needed, streams results back. agent.py prints text blocks as they arrive.

7

State persists

Every meaningful tool call wrote to nucleus_memory.db during the run. Final workflow step stages nucleus_memory.db + data/ subfolders, commits with timestamp, rebases onto origin/main.

8

Push to main

git push lands the new memory state on origin/main. The next workflow that runs (any workflow) gets the updated state at checkout. Concurrency group prevents overlapping pushes.

3. The Python orchestrator (agent.py)

RESPONSIBILITY

agent.py is 198 lines. It does five things: load secrets, register tools, build the prompt, open one SDK session, exit. It contains no business logic. Adding a new workflow means adding two strings (TASK + KICKOFF) — nothing else.

Entry-point shape

```
TASKS = {
    "requirement-management",
    "daily-report-detailed", "daily-report-summary",
    "api-functional-test", "api-integration-test",
    "api-load-test", "e2e-test",
}

TASK_KICKOFF = { # one-line instruction Claude receives first
    "requirement-management":
        "Run the Requirement Management loop now. ...",
    "daily-report-detailed":
        "Build today's Detailed Daily Test Report. ...",
    # ... one entry per task
}

async def run_agent(task, dry_run):
    if dry_run:
        os.environ["NAPCO_NUCLEUS_DRY_RUN"] = "1"
    server = create_sdk_mcp_server(
        name="napco-nucleus", version="0.1.0", tools=ALL_TOOLS)
    allowed = [f"mcp__napco-nucleus__{n}" for n in TOOL_NAMES]
    allowed.extend(["WebSearch", "WebFetch"])
    options = ClaudeAgentOptions(
        system_prompt=_load_prompt(task),
        mcp_servers={"napco-nucleus": server},
        allowed_tools=allowed,
    )
    async with ClaudeSDKClient(options=options) as client:
        await client.query(TASK_KICKOFF[task])
        async for msg in client.receive_response():
            print(_extract_text(msg))
```

WHY ONE TURN PER PROCESS

- **Clean state.** No leaked context between workflows. The 02:00 functional run cannot poison the 09:00 daily-report run.
- **Easy to reason about.** Each run is a function call — input prompt, output side effects. No long-running daemon to babysit.
- **Crash-safe.** Process death between tool calls leaves SQLite in a consistent state because every tool flushes before returning. Restart and re-run.
- **Cost-bounded.** A workflow can't accidentally loop forever — the SDK session closes when the prompt's loop ends, and timeout-minutes: 25 in the YAML caps the worst case.

4. Claude Agent SDK integration

SDK SURFACE NN USES

Three imports do all the work: **ClaudeAgentOptions** (session config), **ClaudeSDKClient** (the session context manager), and **create_sdk_mcp_server** (wraps Python functions as MCP tools). Tools are registered with the **@tool** decorator from the SDK.

How a tool is registered

```
from claude_agent_sdk import tool

@tool(
    "search_requirements", # tool name Claude sees
    "Fuzzy search requirements seen by title. Returns the most "
    "recent N rows with their gitlab_issue_url if present.",
    {"query": str, "limit": int}, # arg schema
)
async def search_requirements_tool(args):
    rows = memory.search_requirements(args["query"],
                                     limit=args.get("limit", 5))
    return {"content": [{"type": "text",
                          "text": json.dumps(rows, default=str)}]}
```

TRANSPORT MODEL — NO ANTHROPIC_API_KEY

The SDK normally talks to `api.anthropic.com` using `ANTHROPIC_API_KEY`. NN does not set that variable. Instead, **napco_config.claude_cli_path()** resolves the path to the locally-installed Claude Code CLI binary, and the SDK is configured with **cli_path=...**. The CLI handles auth via the user's Claude Max subscription session, so every run is metered against the monthly subscription, not billed per token.

Consequence: cost is fixed (~\$200/mo). Consequence: NN cannot run on a CI host that has no Claude Max session. That is why workflows pin to the self-hosted Windows runner, where the CLI is logged in 24/7.

5. The MCP tool surface (31 tools, 6 modules)

MODULE COMPOSITION			
Tools are grouped by responsibility. Each module exposes TOOLS (the registered functions) and TOOL_NAMES (for the allowlist). tools/___init___py concatenates them into ALL_TOOLS so agent.py registers the entire surface in one call.			
Module	N	Responsibility	Representative tools
memory	5	SQLite-backed cross-run continuity	log_activity, recall_activity, search_requirements, recall_test_runs, memory_stats
requirements	4	Email + Drive ingestion → GitLab	poll_requirement_emails, ingest_drive_files, read_requirement_inbox, publish_tasks_to_gitlab
tests	9	Test execution + health probes	run_api_tests, run_integration_tests, run_load_tests, run_e2e_tests, run_single_e2e_test, ...
files	5	Sibling-project file IO + Playwright a11y	list_files, read_file, write_file, edit_file, explore_ui
git	3	Read-only git context across projects	git_diff, git_recent_commits, git_commit_and_push
report	5	PDF + email + Teams + log tail + cleanup	generate_pdf_report, send_email, send_teams_digest, tail_log, clean_reports

TOOL CONTRACT (THE INVARIANTS)

- **Wrap one external system per tool.** No tool calls another tool. Composition is Claude's job, not Python's.
- **Side effects log to memory.** Every tool calls memory.log_activity inside a try/except so a memory write never breaks the primary flow.
- **Honor NAPCO_NUCLEUS_DRY_RUN.** Mutating tools (publish_tasks_to_gitlab, send_email, git_commit_and_push) check the env var and short-circuit.
- **Return the SDK envelope.** Every tool returns {"content": [{"type": "text", "text": json.dumps(...)}]}. Helper _text() in tools/_shared.py enforces this.
- **Lazy heavy imports.** imaplib, googleapiclient, requests for GitLab, Playwright runner all imported inside the tool fn. Keeps cold-start sub-second.

6. The memory layer

WHY SQLITE IN GIT

nucleus_memory.db is committed to main after every workflow. Cloning the project on a new machine instantly recovers prior context: every requirement ever filed, every test run with regressions, every IMAP UID checkpoint, every Drive file ID already processed. No external state store, no Redis, no Postgres. The repo is the database.

Schema (the load-bearing tables)

Table	Purpose
activity_logs	Every meaningful action with task_name (e.g. requirement-management:publish_gitlab), result string, technical_details JSON blob, timestamp. Drives the daily report's 'what NN did' section.
requirements_seen	One row per requirement ever processed. Normalized title plus FTS5 virtual table for fuzzy match. gitlab_issue_iid + gitlab_issue_url populated on success. Powers the dedup that stops duplicate issues across runs.
test_run_history	One row per suite-run: pass/fail counts, duration, PDF artifact path, regression set. Daily report reads this for trend graphs.
email_checkpoints	IMAP UIDVALIDITY + last UID per (host, user) pair. Idempotency key for poll_requirement_emails.
drive_processed	Set of Google Drive file IDs already ingested. Idempotency key for ingest_drive_files.

FTS5 — WHY FUZZY MATCHING MATTERS

Clients describe the same requirement different ways across weeks. Exact title match would file three duplicate GitLab issues for 'Add SSO login', 'Implement single sign-on', and 'Add SSO support'. The FTS5 index over normalized titles plus summary text lets search_requirements catch close variants. publish_tasks_to_gitlab uses this as its second dedup layer — after exact match against currently-open GitLab issues.

7. Prompts — algorithms in markdown

STRUCTURE

Every task gets a system prompt + a task prompt, concatenated at runtime. **prompts/system.md** defines who Claude is (NAPCO Nucleus), the project layout, the two operational dimensions, the core principles (Claude-first, one-shot per run, artifacts over adjectives, mandatory memory check-in), tone rules (no em-dashes, plain dev English, English-only output). **prompts/<task>.md** defines the actual loop for that workflow — numbered steps, required tool calls, decision rules, output format.

The 8 prompt files

File	Workflow it drives
system.md	Concatenated to every task. The shared backbone.
requirement_management.md	Fires every 2h. IMAP+Drive ingest → 3h tasks → GitLab issues.
daily_report_detailed.md	09:00 BDT. 6-section detailed PDF to the full team.
daily_report_summary.md	09:30 BDT. 7-block executive dashboard to leadership.
api_functional_test.md	02:00 BDT. Newman + Postman collection, classify failures.
api_integration_test.md	02:00 BDT. pytest integration suite with regression diff.
api_load_test.md	02:00 BDT. Locust multi-tier 10–10,000 users.
e2e_test.md	02:00 BDT. Playwright full suite + failure screenshots.

WHY THIS WORKS

- **Behavior changes are markdown diffs.** Adjusting how tasks split, how failures classify, what goes in the report — all PR-reviewable as plain English.
- **Constraints encoded in prompts.** 'Memory check-in mandatory.' 'Title under 70 chars.' 'NO em-dashes.' Things that would be brittle validation code become natural-language rules Claude follows.
- **One ground truth per workflow.** If you want to know what API Functional Test does, you read `api_functional_test.md`. The Python tools are leaf operations only.

8. Third-party integrations

Integration	Library	Used for
IMAP	stdlib imaplib	Pull allowlisted requirement emails. Idempotent via UIDVALIDITY + since-UID checkpoint stored in memory.email_checkpoints.
Google Drive	google-api-python-client	List + download new audio/video and PDFs from a configured folder. Service-account auth via google-credentials.json.
Groq Whisper	requests (REST)	Audio + video transcription. Cheap, fast. Result becomes a .txt file under data/requirements/inbox/meetings/.
pypdf	pypdf	Local PDF text extraction. No external call. Result becomes .txt under inbox/documents/.
GitLab	requests (REST v4)	list_open_issue_titles + create_issue. PAT with api scope. Stateless — every call re-reads env so token rotation is hot.
Microsoft Teams	requests	Incoming webhook for the optional one-line digest after requirement-management completes.
Gmail SMTP	stdlib smtplib	Outbound email of the daily report PDFs. App-password auth. Used by send_email tool only.
Newman (Postman CLI)	subprocess	API functional test execution. Reads a Postman collection + environment from MVP-Access-API-Test.
pytest	subprocess	API integration test execution. Test files live in MVP-Access-API-Test/tests/integration.
Locust	subprocess	Load testing in tiers (10, 100, 1k, 10k users). Multi-stage with cooldowns to find the capacity ceiling.
Playwright	subprocess	E2E test execution across Easy / Release / full suites. Failure screenshots embedded in the PDF report.
TFS + MSBuild	subprocess	MVP Access CI/CD workflow only. tf get latest, MSBuild Release config, IIS deploy via UNC, health check.
GitHub Actions	(scheduler)	Cron + manual triggers. 9 workflows. All pinned to runs-on: [self-hosted, Windows].
Self-hosted runner	(machine)	Windows VM at 172.16.205.209. Has the Claude Code CLI logged into Claude Max, Python 3.13, all test runners installed.

9. CI/CD execution model

WORKFLOW SHAPE (UNIVERSAL)

Every NN workflow has the same five steps in YAML: checkout, install, run agent.py with --task, optional follow-on (e.g. log copy), commit + push memory state. Differences are: cron schedule, the --task flag, and which secrets get exposed via env: at job level.

Canonical workflow YAML

```
name: <Display Name>
on:
  schedule:
    - cron: '<utc-cron>'
  workflow_dispatch:
    inputs:
      dry_run:
        type: choice
        options: ['false', 'true']

permissions:
  contents: write

concurrency:
  group: <task-name>
  cancel-in-progress: false

jobs:
  run:
    runs-on: [self-hosted, Windows]
    timeout-minutes: 25
    env:
      # only the secrets this task needs
      ...
    steps:
      - uses: actions/checkout@v5
      - run: py -3 -m pip install -r requirements.txt
      - run: py -3 agent.py --task <task-name>
        shell: pwsh
      - name: Commit state changes
        if: always()
        shell: pwsh
        run: |
          git add nucleus_memory.db data/...
          git commit -m "<task>: $(Get-Date -Format ...)"
          git pull --rebase origin main
          git push
```

OPERATIONAL GUARDRAILS

- **Concurrency groups.** Each workflow declares a group named after its task. cancel-in-progress: false means runs queue, never overlap.
- **timeout-minutes.** 25 minutes for most workflows; load test gets longer. Caps runaway cost in the unlikely event a prompt loops.
- **contents: write.** Required because the post-run step commits memory back to the repo. No other elevated permissions.
- **Self-hosted only.** All workflows pin to runs-on: [self-hosted, Windows]. Cloud runners cannot run NN — no Claude Max session, no Newman/Locust/Playwright pre-installed, no UNC path access for IIS deploy.

10. The 9 workflows

#	Workflow	Schedule (UTC / BDT)	What it does
1	API Functional Test	20:00 / 02:00	Newman + Postman collection across the API surface.
2	API Integration Test	20:00 / 02:00	pytest integration suite with regression diff vs. prior runs.
3	API Load Test	20:00 / 02:00	Locust multi-tier 10 → 10,000 users with server-recovery cooldowns.
4	MVP Access E2E Test	20:00 / 02:00	Playwright full suite. Failure screenshots embedded in PDF.
5	Daily Report (Detailed)	03:00 / 09:00	Reads memory + the 4 test PDFs. Composes 6-section detailed PDF. Emails full team.
6	Daily Report (Summary)	03:30 / 09:30	7-block executive dashboard. Emails leadership only.
7	Requirement Management	Every 2h, 09–17 BDT, Sun–Thu	IMAP + Drive ingest. Splits into 3-hour tasks. Files GitLab issues with two-layer dedup.
8	MVPAccess CICD	16:00 / 22:00	TFS pull, MSBuild Release, IIS deploy via UNC, health check, memory log.
9	Probe Runner Filesystem	Manual	Diagnostic. Inspects runner state during triage.

TWO OPERATIONAL DIMENSIONS

Project Management dimension. Workflows 5, 6, 7. Read state from the world (email, Drive, prior runs), produce reports + GitLab issues. Where strategy meets execution.

Test Automation dimension. Workflows 1, 2, 3, 4, 8. Run suites, classify failures, deploy releases. Pure execution.

11. Adding a new workflow

RECIPE

1

Write the prompt

Create prompts/<new_task>.md. Define the loop: memory check-in (mandatory), the steps, the dedup rule, the output contract. Match tone of existing prompts.

2

Wire agent.py

Add the task name to TASKS set. Add a one-line kickoff message to TASK_KICKOFF dict. No other Python changes if existing tools cover the I/O.

3

If new I/O is needed

Add a tool — see section 12. Otherwise skip.

4

Author the workflow YAML

Copy an existing workflow file in .github/workflows/. Update name, cron, env: secrets the task needs, the --task flag, and the post-run commit pattern.

5

Add secrets

If the task needs new env vars, add them to GitHub Actions secrets AND to the runner's local .env. agent.py loads .env with override=True so .env wins for local runs.

6

Dry-run first

Trigger via workflow_dispatch with dry_run=true. Verify the loop logs to memory but does not mutate. Inspect activity_logs.

7

First scheduled run + monitor

Switch to live. Tail logs from the runner. Confirm the post-run commit landed on main with the new state.

12. Adding a new MCP tool

RECIPE

1

Pick the right module

memory / requirements / tests / files / git / report. If none fit, create tools/<new>.py with the same shape.

2

Write the function

@tool decorator with name, description, arg schema. Lazy-import any heavy library inside the fn. Wrap external call in try/except. Always log_activity. Honor NAPCO_NUCLEUS_DRY_RUN if mutating.

3

Return the SDK envelope

Use _text(payload) helper from tools/_shared.py. It produces {"content": [{"type": "text", "text": json.dumps(payload, default=str)}]}.

4

Append to TOOLS + TOOL_NAMES

Both lists at the bottom of the module file. tools/__init__.py picks them up automatically — no edits there.

5

Update the prompt that calls it

Add the tool name + when to call it to the relevant prompts/<task>.md. Without this, Claude does not know the tool exists.

6

Smoke-test locally

py -3 agent.py --task <task> --dry-run from the repo root. Watch the streamed text for the tool call result.

TOOL TEMPLATE

```
from claude_agent_sdk import tool
import memory
from tools._shared import _text

@tool(
    "do_thing",
    "What it does, in one sentence Claude can act on.",
    {"arg1": str, "arg2": int},
)
async def do_thing_tool(args):
    if os.environ.get("NAPCO_NUCLEUS_DRY_RUN") == "1":
        return _text({"dry_run": True})
    try:
        result = backend.run(args["arg1"], args["arg2"])
    except Exception as e:
        memory.log_activity(task_name="do_thing",
                           result=f"error:{type(e).__name__}")
        return _text({"error": str(e)})
    memory.log_activity(task_name="do_thing",
                       result=f"ok:{result.summary}")
    return _text(result.to_dict())

TOOLS = [do_thing_tool]
TOOL_NAMES = ["do_thing"]
```

13. Benefits

WHAT THIS ARCHITECTURE BUYS

- **Behavior-via-prompt.** Adjusting how requirements are split, how failures classified, what shows up in the executive summary — all happen in markdown PRs. No deploy, no release.
- **Cost-bounded reasoning.** Claude Max subscription is fixed monthly. Per-token API billing is not in the budget. NN's cost ceiling does not depend on traffic.
- **Single-machine deployment.** One self-hosted runner, one repo, one .env. No K8s, no message queue, no service mesh. The whole platform fits in one VM and one git repo.
- **Memory in git.** Audit trail, dedup history, run history all version-controlled. Forensic question 'what did NN file last Tuesday?' answered with a git log + a sqlite query.
- **Idempotent by construction.** Every ingestion path has a checkpoint or file-ID. Every publish path has two-layer dedup. Re-running any workflow is safe.
- **Tool surface stays small.** 31 tools across 6 modules cover both dimensions. Reasoning logic that would inflate Python LOC lives in markdown.
- **One Claude turn per process.** No session leak, no context bleed, no need to manage long-lived agent state. Process boundary IS the state boundary.
- **Standard CI/CD.** Workflows are vanilla GitHub Actions YAML. Anyone who knows Actions can read a NN workflow without learning a new orchestrator.
- **Multi-language input.** Bangla, Malay, mixed-script chat — Claude reads it. Outputs are always English by prompt rule. No translation pipeline needed.
- **Two consolidated emails per day.** 09:00 detailed report to the team, 09:30 executive summary to leadership. Replaces the previous 6-fragments-overnight reality.

14. Demerits and known limitations

WHAT THIS ARCHITECTURE COSTS

- **Single-machine bottleneck.** Self-hosted runner is a single VM. If it goes down, every workflow stops until it's back. No automatic failover.
- **Claude Max session is a hidden dependency.** If the Claude Code CLI on the runner logs out, every workflow fails until somebody re-authenticates interactively. There is no headless retry path.
- **Non-determinism.** Same input does not guarantee same output. Two consecutive runs may classify the same failure slightly differently. Acceptable for reports, problematic for invariants.
- **Prompt drift.** A small markdown edit can change behavior subtly across all consumers. There is no automated regression suite for prompt changes — only dry-run and manual review.
- **Memory growth.** `nucleus_memory.db` grows monotonically. No retention policy yet. Repo size will eventually feel it. Vacuum or pruning is on the roadmap.
- **Tooling-language coupling.** Tools are Python. Tests are TypeScript (Playwright) and JS (Newman). Crossing that boundary is via subprocess, not in-process — debugging is log-driven.
- **Secrets fan-out.** Every secret has to live in two places: GitHub Actions secrets (for the workflow env) and the runner's `.env` (for local re-runs). Rotation is a two-step manual process.
- **Limited concurrency.** Only one job at a time per workflow group. The runner cannot parallelize the four 02:00 test runs — they queue on the same machine.
- **No multi-tenant story.** Allowlists, project paths, GitLab project ID are hard-coded for MVP Access. Onboarding a second product would require config refactoring.
- **Hard-to-test reasoning.** Unit tests cover tools (I/O wrappers). The reasoning lives in prompts. Validating a prompt change is observational, not assertive.

15. Tech stack at a glance

EVERYTHING NN DEPENDS ON

Reasoning: Claude Agent SDK · Claude Code CLI · Claude Max subscription.

Runtime: Python 3.13 · GitHub Actions · self-hosted Windows VM runner.

Memory: SQLite + FTS5. Committed to git.

Reporting: Reportlab.

Tests it orchestrates: Newman · pytest · Locust · Playwright.

External integrations: IMAP · Gmail SMTP · Google Drive API · Groq Whisper · GitLab REST v4 · Microsoft Teams webhook · TFS · MSBuild · IIS.

Python deps: claude-agent-sdk · python-dotenv · requests · google-api-python-client · google-auth · pypdf.

16. Closing

THE THESIS, RESTATED

QA architect plus AI equals senior developer team output. NAPCO Nucleus is the proof: 9 production workflows, 31 tools, ~3.6k Python LOC, two consolidated emails per day, every client requirement filed in GitLab within two hours of being spoken on a call. Built and operated by one person, because the reasoning lives in prompts, not in code that needs a team to maintain.